

Sistema de agendamento médico

**Equipe: João Guilherme, Marcondes Simão,
Milene Gonçalves, Wellington Gois**

Objetivos do sistema:

- Validar o login do usuário;
- Gerar prontuário;
- Atualizar status do atendimento;
- Usuário realizar cadastro/login;
- Usuário solicitar e acompanhar atendimento;
- Listar os atendimentos agendados;
- Sanar a ‘dor’ do médico e auxiliar no gerenciamento da rotina da clínica;

Documentação do projeto

Diagrama de classes

Usuário é a classe base, tem nome, email, senha e perfil.

Paciente e **Médico** herdam de Usuário (ou seja, aproveitam seus atributos e adicionam os próprios).

- Paciente tem CPF, telefone e data de nascimento.
- Médico tem CRM e métodos para visualizar fichas e prescrever medicação.

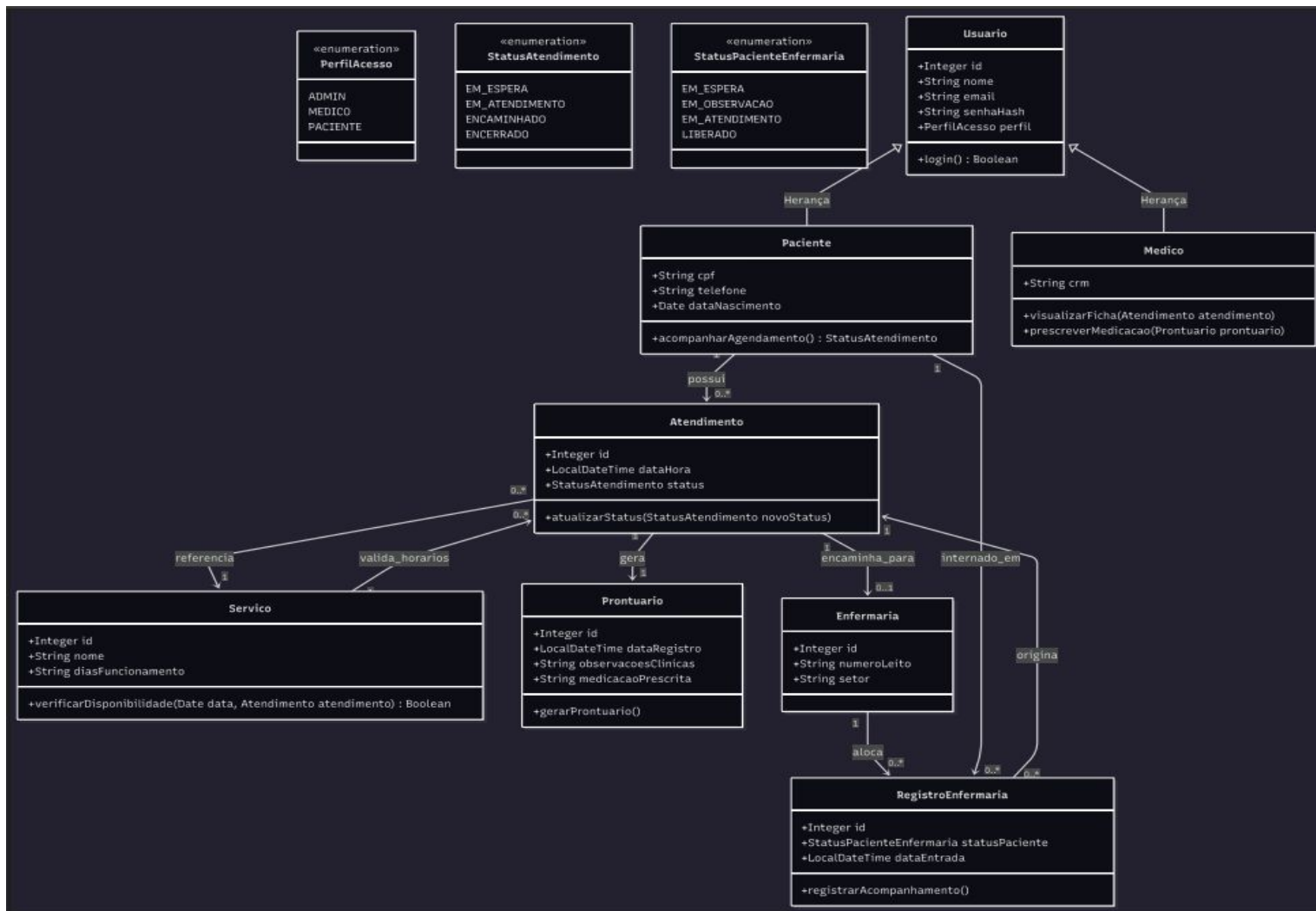


Diagrama de Classes

Atendimento

Representa o encontro entre paciente e médico.

- Guarda **data/hora**, **status** e tem métodos para **atualizar o status**.
- Está ligado a **Paciente**, **Médico**, **Serviço**, **Prontuário** e **Enfermaria**.

Prontuário

Guarda informações clínicas:

- Data de registro, observações e medicação prescrita.
- Método para **gerar o prontuário**.

Enfermaria e RegistroEnfermaria

- **Enfermaria** tem número do leito e setor.
- **RegistroEnfermaria** guarda o status do paciente, data de entrada e acompanhamento.

Serviço

Define os serviços disponíveis

- Nome, dias de funcionamento e método para **verificar disponibilidade**.

Relações entre classes

- **Usuário** → **Paciente/Médico**: herança.
- **Paciente** → **Atendimento**: o paciente “possui” atendimentos.
- **Atendimento** → **Prontuário**: gera o prontuário.
- **Atendimento** → **Enfermaria**: encaminha o paciente.
- **Enfermaria** → **RegistroEnfermaria**: registra entrada e acompanhamento.
- **Serviço** → **Atendimento**: valida horários e disponibilidade.

Diagrama de atividades

Início e acesso

O processo começa com o login.

- Se o usuário não tem cadastro, ele precisa **se registrar**.
- Se o login der erro, o sistema mostra uma mensagem de erro.
- Quando o login é válido, o sistema **identifica o perfil** do usuário (paciente, médico ou administrador).

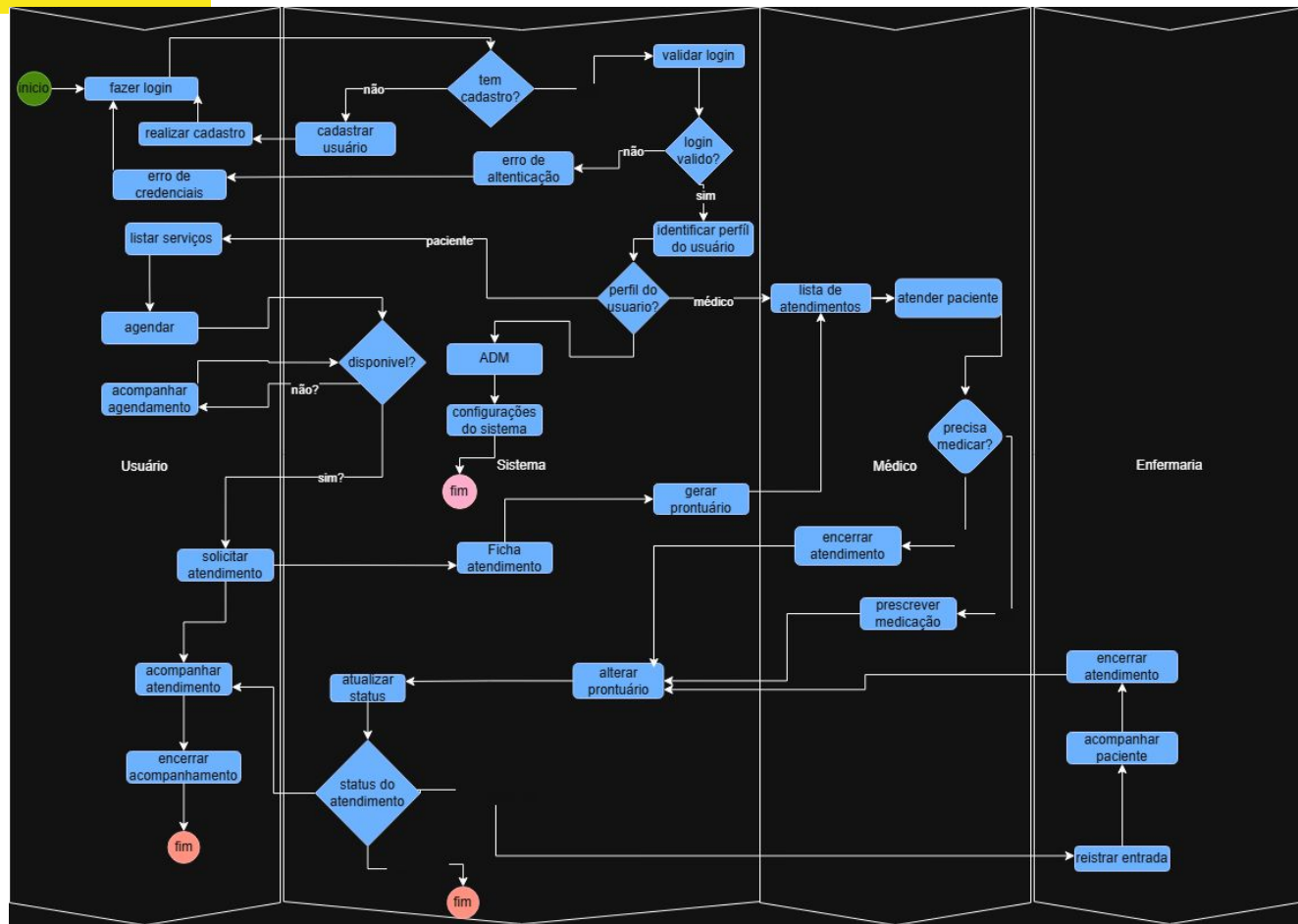


Diagrama de atividades

Usuário (paciente)

- **Listar serviços** disponíveis.
- **Agendar** um atendimento e **acompanhar** esse agendamento.
- Se houver disponibilidade, ele pode **solicitar atendimento**.
- Durante o atendimento, o sistema **atualiza o status** e o paciente pode **acompanhar** até o **encerramento**.

Enfermaria

- **Registra a entrada** do paciente.
- **Acompanha** o tratamento e **encerra o atendimento** quando finalizado

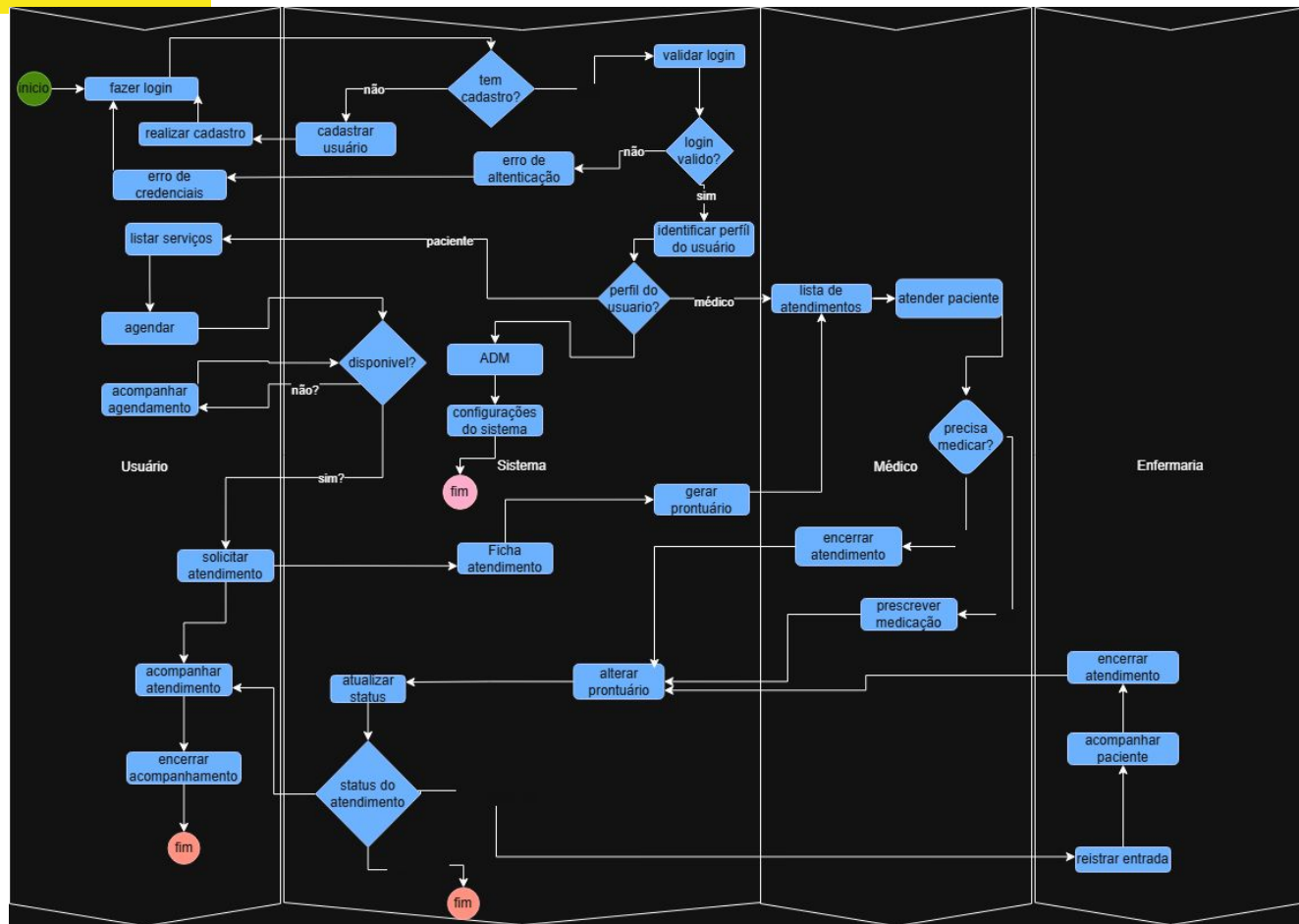


Diagrama de atividades

Sistema

O sistema é responsável por:

- **Gerar a ficha de atendimento** (prontuário).
- **Atualizar status e alterar prontuário** conforme o atendimento evolui.
- Encerrar o processo quando tudo estiver concluído.

Médico

- **Vê a lista de atendimentos.**
- **Atende o paciente e decide se é necessário prescrever medicação.**
- **Depois, encerra o atendimento.**

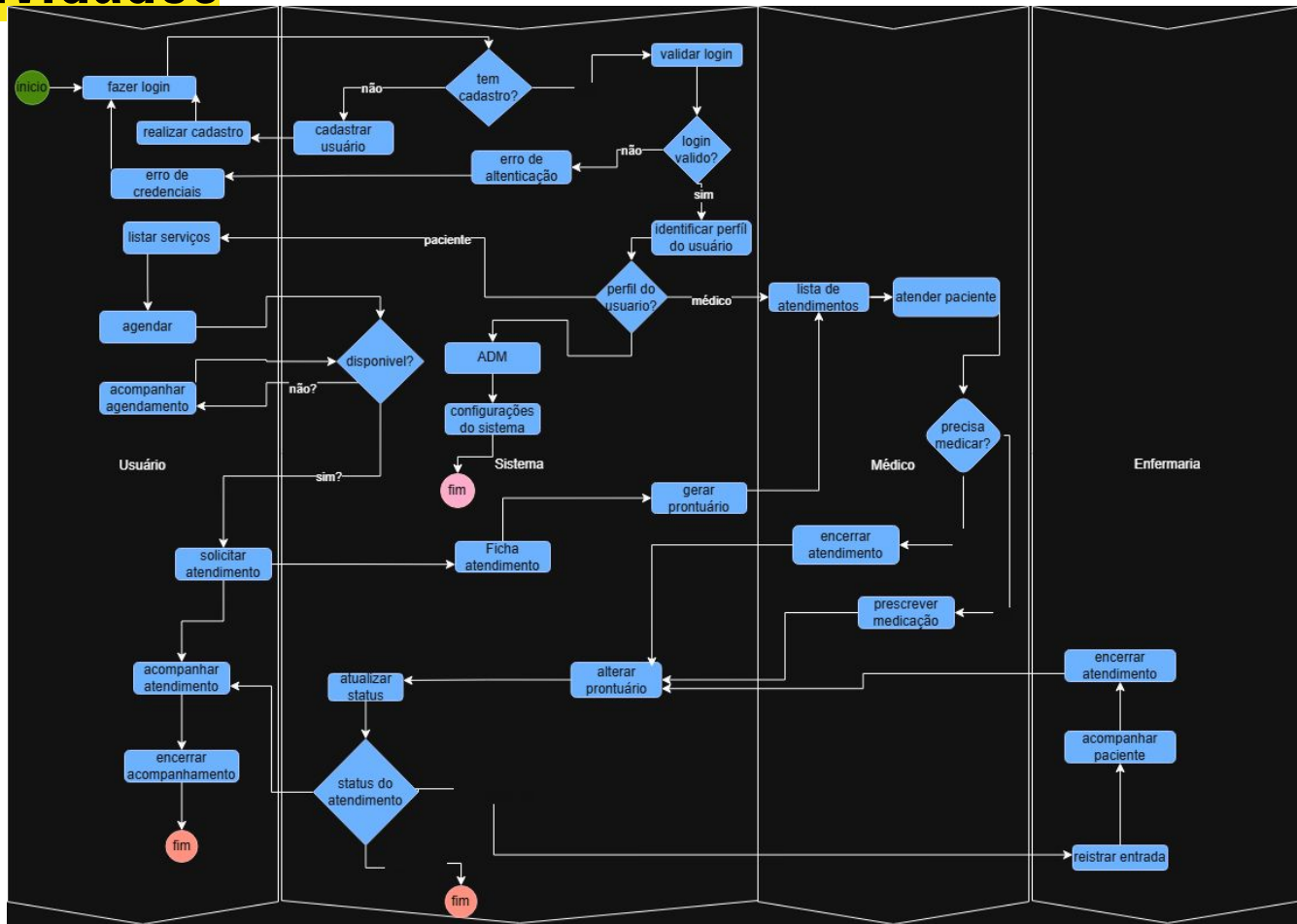
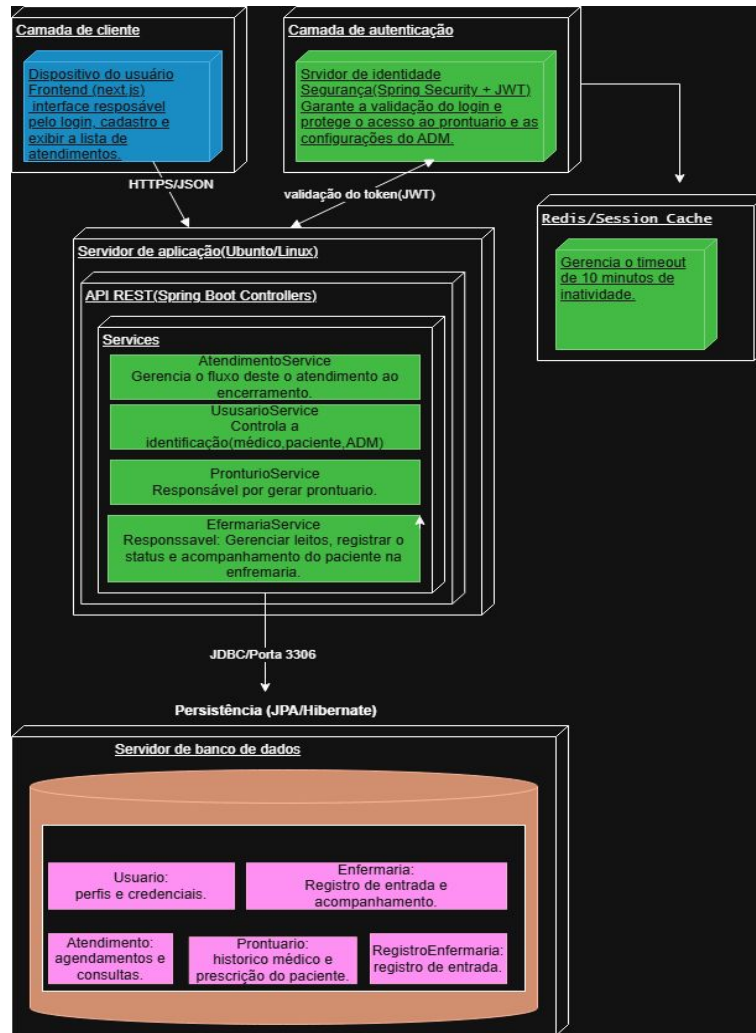


Diagrama de implementação

- Mostra como o sistema hospitalar é estruturado, desde o acesso do usuário até o armazenamento dos dados.
- Frontend feito com **Next.js**.
- Responsável por **login, cadastro e exibição dos atendimentos**.
- Usa **Spring Security + JWT** para validar o acesso. Garante que só quem tem permissão entre no sistema (paciente, médico ou administrador).
- **Redis / Sessão Cache** Se o usuário ficar inativo por mais de **10 minutos**, a sessão expira por segurança.
- Servidor de aplicação usa **Spring Boot** com **API REST** para comunicação entre o frontend e o banco de dados.



Banco de dados

- **Diagrama** representa como as informações de um sistema hospitalar estão organizadas e se relacionam.
- Um **usuário** pode ser um **paciente**.
- Um **paciente** faz um **atendimento**, que é realizado por um **médico** e está ligado a um **serviço**.
- Cada **atendimento** gera um **prontuário** (registro médico).
- A **enfermaria** recebe o paciente e cria um **registro de entrada**.

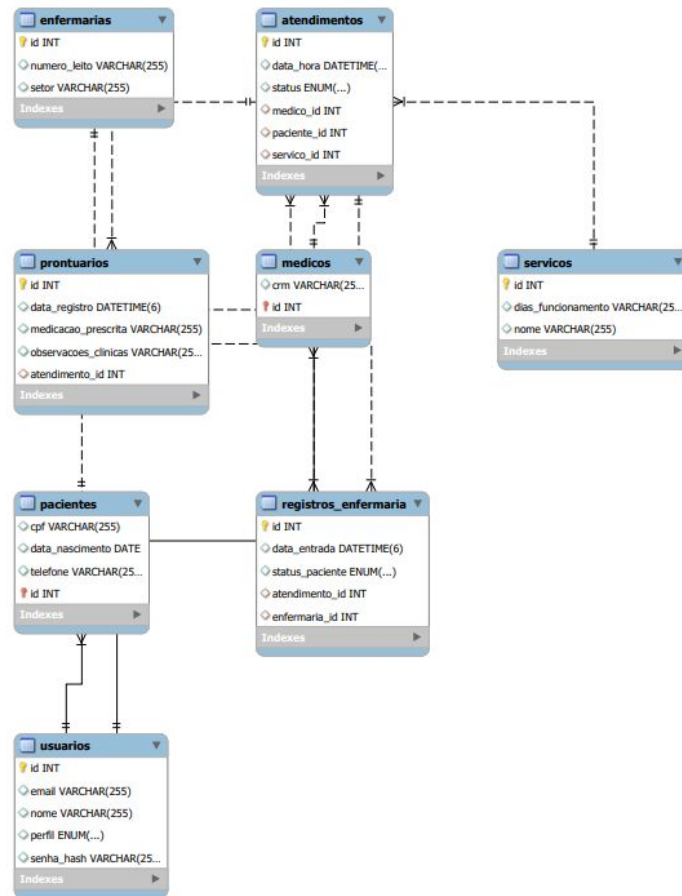


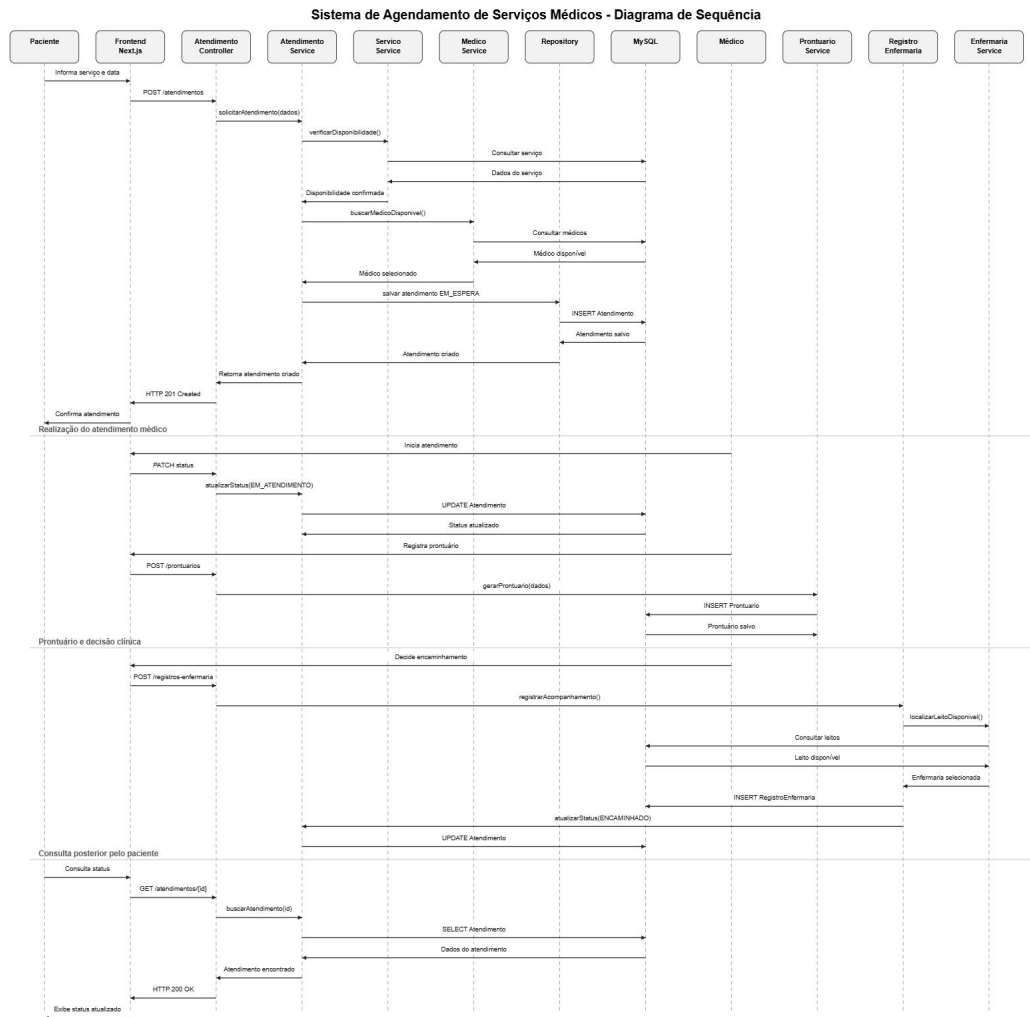
Diagrama de sequencia

O processo começa com o **paciente** informando o serviço que deseja.

- O **frontend (Next.js)** envia um pedido **POST /atendimento** para o **AtendimentoController**.
- O **AtendimentoService** verifica se o serviço está disponível (**verificaDisponibilidade()**).
- O **ServicoService** consulta os dados do serviço e confirma a disponibilidade.
- O atendimento é criado com o status **EM_ESPERA** e salvo no banco de dados (**INSERT Atendimento**).
- O sistema responde com **HTTP 201 Created**, confirmando que o agendamento foi feito.

Quando o médico inicia o atendimento:

- O sistema atualiza o status para **EM_ATENDIMENTO** (**PATCH /status**).
- O **AtendimentoService** faz o **UPDATE** no banco.
- Depois, o médico registra o **prontuário** (**POST /prontuario**), com observações e medicação.
- O **ProntuarioService** gera e salva o registro (**INSERT Prontuario**).



Sistema de Agendamento de Serviços Médicos - Diagrama de Sequência

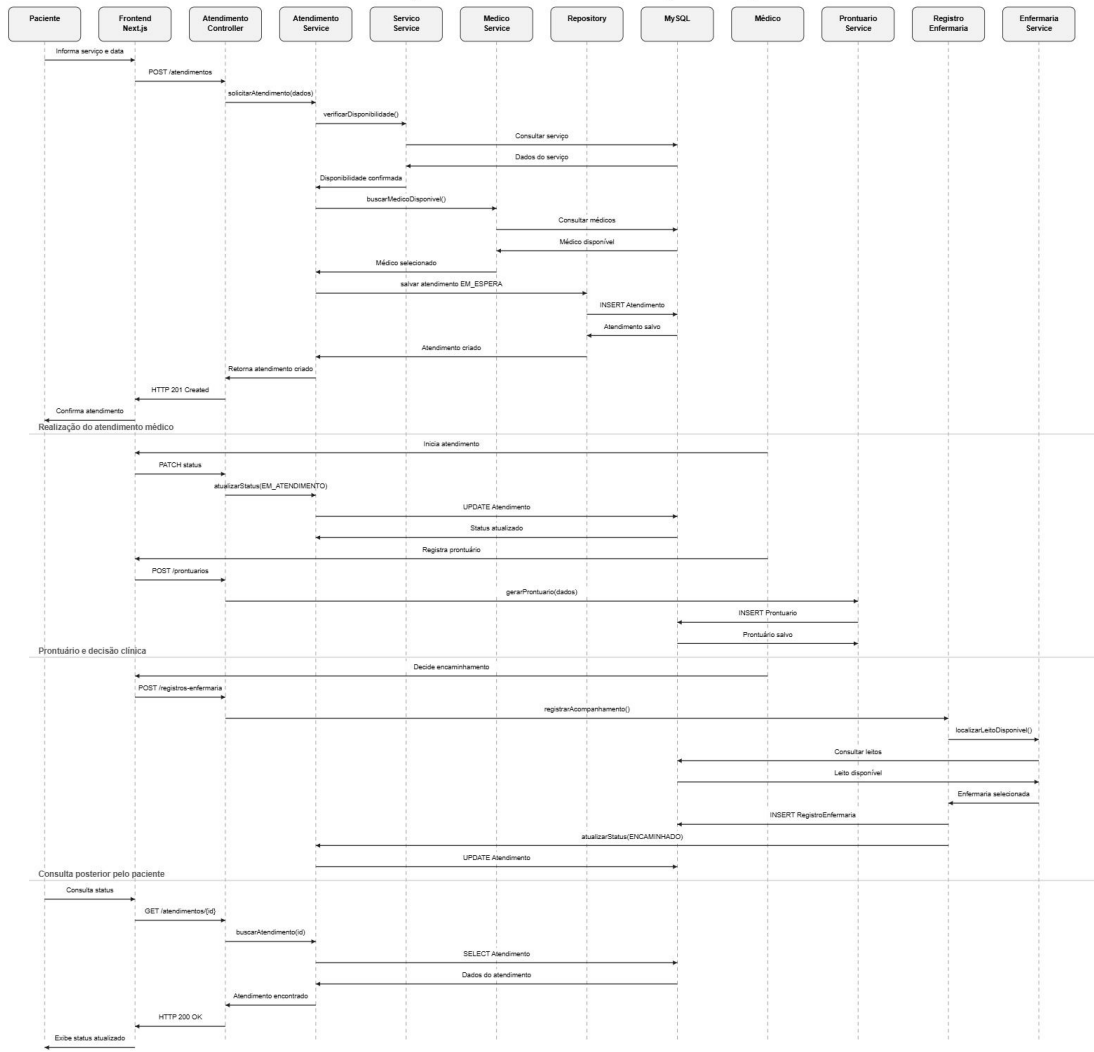


Diagrama de sequencia

Se o paciente precisar ser internado ou observado:

- O médico decide o encaminhamento (`POST /registro-enfermaria`).
- O **RegistroEnfermariaService** registra o acompanhamento e consulta os **leitos disponíveis**.
- A **EnfermariaService** seleciona o leito e salva o registro (`INSERT RegistroEnfermaria`).
- O status do atendimento muda para **ENCAMINHADO**.

Depois do atendimento, o paciente pode consultar o status:

- O frontend faz um `GET /atendimento/{id}`.
- O **AtendimentoService** busca os dados (`SELECT Atendimento`).
- O sistema retorna **HTTP 200 OK** com o status atualizado.

API

Arquitetura e comunicação do funcionamento da API do projeto

A API do Sistema de Agendamento Médico atua como o "cérebro" do backend. Ela é a ponte de comunicação que permite que a interface visual (o frontend em Next.js) interaja de forma segura com o nosso banco de dados (MySQL), processando todas as regras de negócio exigidas.

A API foi construída seguindo o padrão RESTful, o que significa que ela é organizada, previsível e utiliza as rotas e métodos padrão da web para transacionar dados em formato JSON.

Papel dos controllers

1. **1. O Papel dos Controllers (Controladores)**
2. **Na arquitetura, o código do backend está dividido em Controllers. Cada Controller atua como um "gerente" responsável por escutar e responder às requisições de uma área específica do sistema (entidades).**
3. **UsuarioController: É o porteiro do sistema. Ele gerencia as rotas de cadastro e a segurança, validando o e-mail e a senha criptografada para emitir o Token JWT de login.**
4. **PacienteController e MedicoController: Gerenciam todo o fluxo de dados cadastrais (como CPF, CRM, contatos) dos atores do sistema.**
5. **AtendimentoController: É o núcleo operacional. Ele recebe os pedidos de novos agendamentos, cruza os dados do médico com o paciente e gerencia a mudança de status (ex: "Em Espera" para "Atendido").**
6. **EnfermariaController e ProntuarioController: Lidam com o módulo de internação e registros clínicos após o atendimento.**

Como funcionam os métodos:

Como funcionam os Métodos HTTP (GET e POST)

Para que o frontend peça algo para os Controllers, ele utiliza "verbos" HTTP. Cada verbo representa uma intenção de ação no banco de dados. No nosso MVP, destacam-se duas operações fundamentais:

GET (Buscar/Ler Dados): Sempre que o usuário abre uma tela que precisa exibir informações, o frontend dispara uma requisição GET. O GET nunca altera nada no banco de dados, ele apenas "puxa" a informação.

Exemplo prático: Quando o administrador acessa a tela de Médicos, o sistema faz um GET /api/medicos. O MedicoController recebe o pedido, vai até o banco de dados, pega a lista de todos os médicos cadastrados e devolve essa lista em formato JSON para o frontend renderizar a tabela na tela.

POST (Criar/Enviar Dados):

Sempre que o usuário preenche um formulário e clica em "Salvar" ou "Agendar", o frontend dispara uma requisição POST. O POST carrega um pacote de dados (JSON) no seu corpo para ser processado e salvo no banco.

Exemplo prático do funcionamento no código:

Ao agendar uma consulta, o frontend faz um POST `/api/atendimentos`, enviando um JSON com o ID do Médico e o ID do Paciente. O `AtendimentoController` recebe esse pacote, aplica as regras de negócio (verifica se os IDs existem), cria a ficha de atendimento com o status inicial e salva a nova linha no banco de dados, devolvendo uma mensagem de sucesso.

Resumo do Fluxo de Funcionamento

O usuário interage com a interface (Next.js).

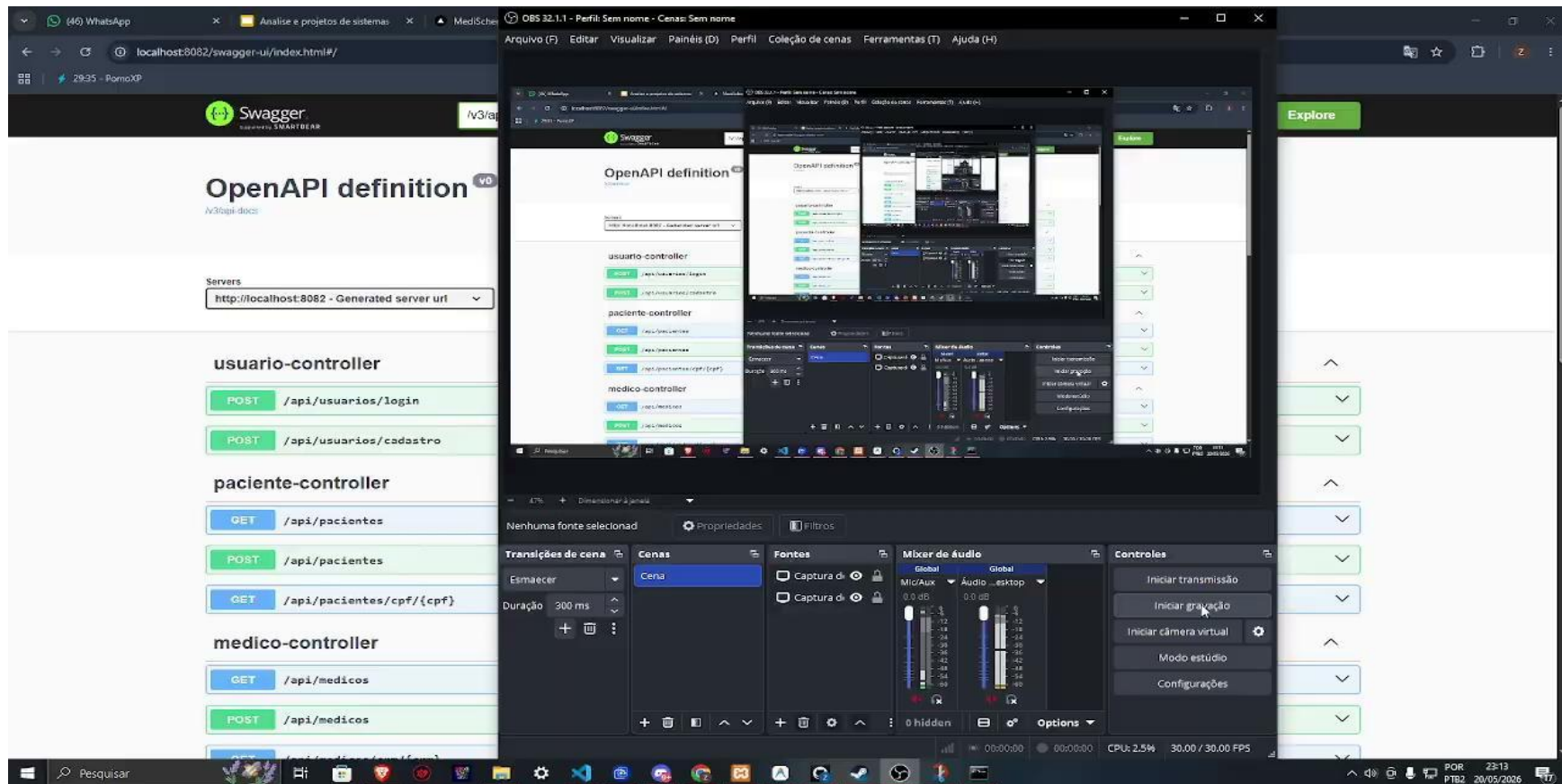
O sistema dispara uma requisição (GET para ler ou POST para salvar), anexando o Token JWT por segurança.

A requisição bate na porta do Controller correspondente no backend (Spring Boot).

O Controller processa, acessa o banco de dados e devolve uma resposta (ex: 200 OK com os dados solicitados).

O frontend recebe a resposta e atualiza a tela para o usuário em tempo real.

Vídeo demonstrando o funcionamento da API

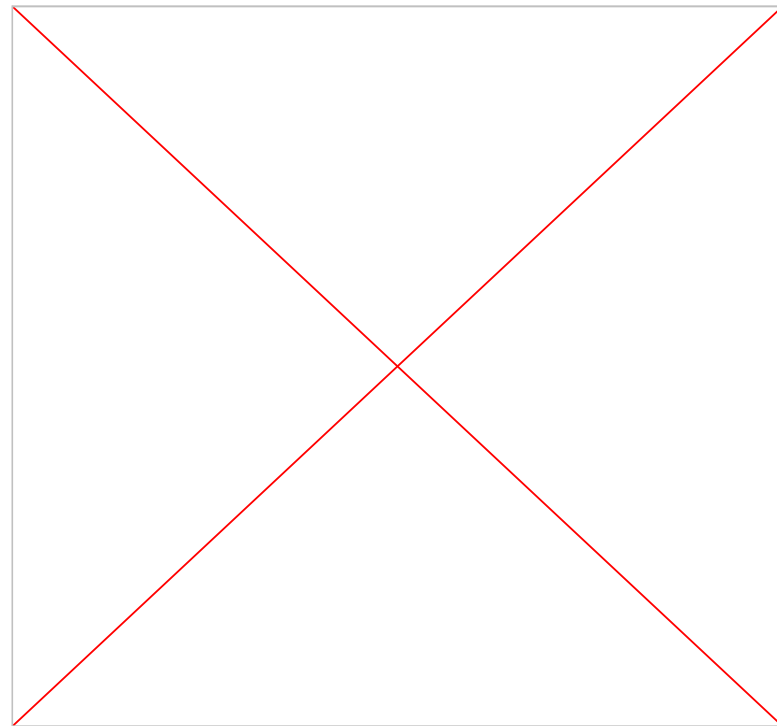


Testes

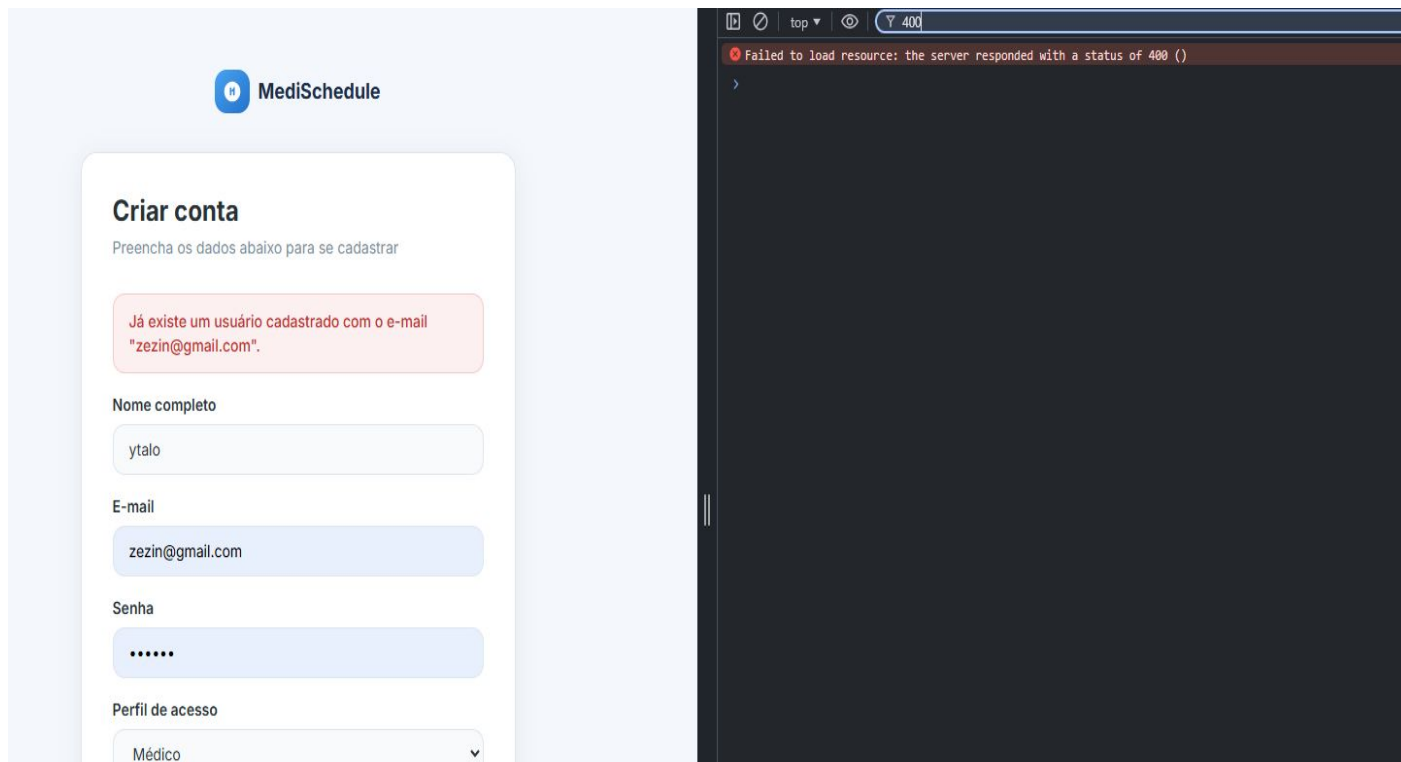
Testes de login e erros de segurança

Código retorna erro 403 e erro 401 ao tentar realizar login com credenciais erradas;

Login com credenciais válidas, sucesso ao entrar no sistema;



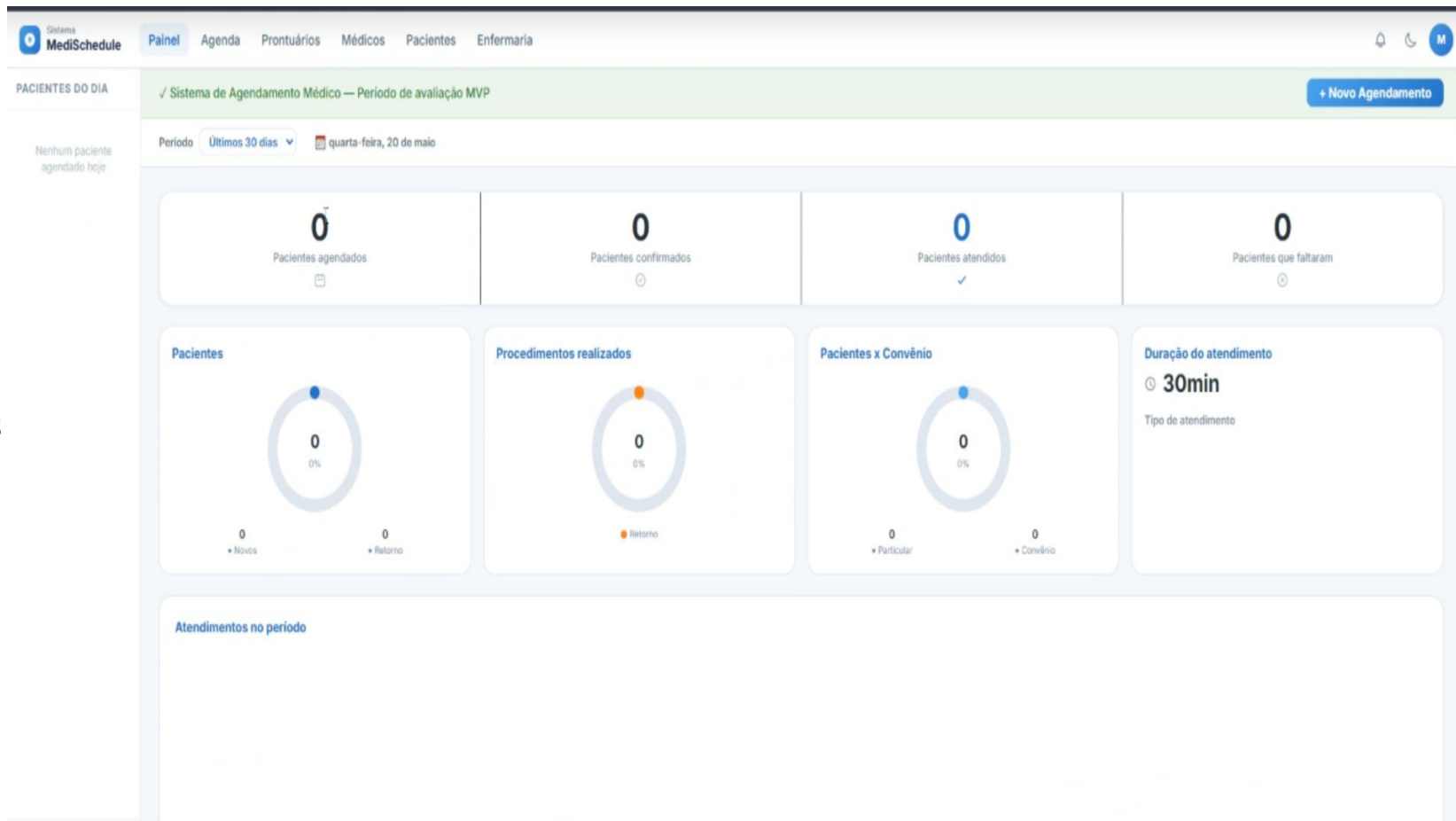
Ao inserir email duplicado código retorna erro 400.



O sistema

Página inicial ao fazer login

Página inicial do sistema, mostrando pacientes agendados e atendidos no dia, procedimentos feitos e comparativo entre convênio e particular.



Vídeo teste mostrando o sistema e suas funcionalidades

